ROOTCAUSE SERVER

06 · Explicación profunda del código

Documento: 06-deep-code-explanation.md **Versión analizada:** 0.2.0 **Commit:** 2efedb2 **Fecha del análisis:** 2026-08-26

CONTENIDO

1 · rootcause-core::security — clasificar lo observado
- Objetivo
- BindScope::classify — la función más importante del producto
- SecuritySignals — agregados y brechas

2 · rootcause-core::policy — los números, validados
- validate — una política que no puede dispararse es peor que ninguna
- exposure_severity — el alcance multiplica el riesgo

3 · rootcause-core::detect — el motor
- DetectionEngine::analyze — flujo completo
- merge_by_fingerprint — por qué existe
- fingerprint — la clave de deduplicación
- exposure::detect — bucle sobre los sockets expuestos
- intrusion::authentication — de eventos a presión
- intrusion::egress — línea base propia del host
- integrity::detect — dos reglas por archivo
- hygiene::detect y hygiene::clock
- resource::cpu — una racha, no un pico
- resource::hours_to_full y runway
- availability::agent_silence — la regla que corre en el servidor

4 · rootcause-core::posture — un número con su letra pequeña

5 · rootcause-core::runbook — escribir, nunca ejecutar

6 · rootcause-server::defense — el perímetro propio
- Por qué está en memoria
- check — cubeta de tokens, paso a paso

7 · rootcause-server::auth — el orden es la característica

8 · rootcause-server::api::ingest_telemetry — el ciclo completo

9 · rootcause-server::storage — SQLite con la evidencia intacta
- connect
- upsert_incident — las reglas de negocio del ciclo de vida
- auto_resolve
- list_incidents
- purge
- threat_severity — la escala de la vista de amenazas

10 · rootcause-server::watchdog — lo que pasa cuando nadie habla

11 · rootcause-server::headers y ui — la consola no se puede armar

12 · rootcause-agent::probe — la promesa auditable

13 · rootcause-agent::net — tres plataformas, un contrato

14 · rootcause-agent::authlog — contar sin copiar

15 · rootcause-agent::integrity — huella sí, contenido no

16 · rootcause-agent::main — transporte y reintento

17 · console/app.js — construir el DOM a mano

18 · Los guardianes de scripts/

> Cómo funciona por dentro cada módulo relevante: objetivo, entrada y salida, flujo interno, decisiones condicionales, bucles y transformaciones, llamadas a otras funciones, cambios de estado, acceso a

> archivos, red o base de datos, tratamiento de errores, reglas de negocio y casos límite.
>
> Los fragmentos de código son mínimos y siempre indican archivo y símbolo. Las funciones triviales (`as_str` , `label` , `rank`) se documentan en bloque; las que concentran una decisión de producto se explican paso a paso.

---

1 · `rootcause-core::security` — clasificar lo observado

Objetivo

Describir **lo que el agente vio**, nunca lo que el servidor decidió. La separación es lo que permite reevaluar un incidente meses después contra la misma evidencia.

`BindScope::classify` — la función más importante del producto

Entrada: una dirección de enlace como texto, tal cual la escribió `ss` , `netstat` o `/proc/net/tcp` . **Salida:** `Loopback` , `Private` o `Public` .

Flujo, bloque por bloque (`security.rs:59`):

1. **Normalización.** Recorta espacios y quita los corchetes de un literal IPv6: `[::1]` y `::1` deben clasificar igual.
2. **Comodín.** `*` o cadena vacía → `Public` . Así es como varias herramientas escriben «todas las interfaces».
3. **Parseo.** Si `IpAddr::from_str` falla, **el resultado es** `Public` :

``` `rust IpAddr::from_str(trimmed).map_or(Self::Public, Self::classify_ip)` ```

Esta línea es una decisión de producto, no un descuido. Ante evidencia ambigua se reporta el caso peor; degradar un hallazgo porque no se entendió la dirección sería exactamente el error que un producto de seguridad no puede cometer. La prueba `unparseable_addresses_fail_towards_the_worse_case` lo fija.

1. **Clasificación de la IP** (`classify_ip`):
   - loopback → `Loopback` ;
   - no especificada (`0.0.0.0` , `::`) → `Public` ;
   - IPv4: privada RFC1918, link-local, o CGNAT `100.64.0.0/10` → `Private` ;
   - IPv6: `fc00::/7` (unique-local) o `fe80::/10` (link-local) → `Private` ;
   - cualquier otra → `Public` .

Caso límite conocido: una dirección pública real y una dirección ilegible producen el mismo veredicto. El operador ve el endpoint textual en la evidencia, así que puede distinguirlas a ojo.

`SecuritySignals` — agregados y brechas

Cuatro métodos hacen el trabajo:

Método	Transformación
`distinct_peers()`	Recolecta las direcciones remotas en un `BTreeSet` y cuenta
`failures_by_source()`	Recorre `auth_events` , **descarta los éxitos** y suma `count` por dirección
`exposed_listeners()`	Itera y filtra los que **no** son loopback
`has_gap(surface)`	Busca una brecha por nombre de superficie
```

`failures_by_source` ignora los éxitos a propósito: la ráfaga y el acceso concedido son dos reglas distintas, y mezclarlas escondería la segunda.

---

2 · `rootcause-core::policy` — los números, validados

`validate` — una política que no puede dispararse es peor que ninguna

Objetivo: rechazar al arrancar una política contradictoria. El comentario del código lo dice sin rodeos: *«looks like defence and provides none»*.

Comprobaciones, en orden (`policy.rs:117`):

1. Para CPU, memoria y disco: ambos umbrales dentro de 0–100, y `high` no mayor que `critical` .
2. `cpu_sustained_samples ≥ 1` . Con `0` la regla de CPU nunca se evaluaría bien.
3. `disk_runway_hours > 0` .
4. `failures_per_source_high ≥ 1` y no mayor que `failures_per_source_critical` .
5. `peer_fanin_high` no mayor que `peer_fanin_critical` .
6. `egress_multiplier > 1.0` . Un multiplicador de `0.5` dispararía la regla de salida en cualquier tráfico normal.
7. `silence_intervals ≥ 1` y `clock_skew_seconds > 0` .

Errores: `Result<(), String>` con el mensaje exacto. `main.rs` lo convierte en un fallo de arranque; `rootcause-server policy --file` lo usa para validar antes de desplegar.

`deny_unknown_fields` en los cuatro `struct` hace que un campo mal escrito (`resourc` en vez de `resource`) sea un error de arranque en vez de un valor ignorado en silencio.

`exposure_severity` — el alcance multiplica el riesgo

```
let base = match class {
    PortClass::Database | PortClass::Infrastructure => Severity::Critical,
    PortClass::RemoteAdmin | PortClass::Cleartext    => Severity::High,
    PortClass::FileShare                             => Severity::High,
    PortClass::Mail | PortClass::Web                 => Severity::Medium,
    PortClass::Other                                 => Severity::Low,
};
```

Después:

- `Loopback` → **retorno inmediato** de `Severity::Info` . Un servicio que solo alcanza el propio host no es superficie.
- `Public` → severidad base sin cambios.
- `Private` → `downgrade(base)` , un escalón menos.
- **Excepción que gana a todo:** si el rol es `DatabaseServer` , el alcance es `Public` y la clase no es `Other` , el resultado es `Critical` .

Y antes de todo esto, en el detector, actúa `is_allowed_public` : un servidor de borde *puede* publicar 80 y 443, más lo que añada `public_allowlist` . Tratar eso como hallazgo entrenaría al operador a ignorar la vista de

superficie entera.

---

3 · `rootcause-core::detect` — el motor

`DetectionEngine::analyze` — flujo completo

```
exposure::detect(&self.policy, input, &mut findings);
intrusion::detect(&self.policy, input, &mut findings);
integrity::detect(&self.policy, input, &mut findings);
hygiene::detect(&self.policy, input, &mut findings);
resource::detect(&self.policy, input, &mut findings);
let mut findings = merge_by_fingerprint(findings);
findings.sort_by(/* severidad descendente, luego huella ascendente */);
```

Cada familia **acumula en el mismo vector**, sin conocer a las demás. El orden de invocación no afecta al resultado, porque la ordenación final es determinista: severidad descendente y, a igualdad, huella ascendente. Esa segunda clave es lo que hace que dos ejecuciones con la misma entrada produzcan exactamente la misma lista, en el mismo orden.

`merge_by_fingerprint` — por qué existe

Un servicio enlazado a la vez en `0.0.0.0` y en `::` produce dos candidatos con la misma huella. Sin fusión, el contador de ocurrencias avanzaría dos veces por ciclo y **un operador leería una condición estable como una escalada**.

La fusión conserva:

- si el nuevo candidato es más grave, **reemplaza** al existente pero antepone la evidencia anterior;
- si no, añade solo la evidencia que aún no estaba presente.

`fingerprint` — la clave de deduplicación

```
if key.is_empty() { format!("{agent}:{rule_id}") }
else { format!("{agent}:{rule_id}:{key}") }
```

El `key` distingue instancias dentro de la misma regla: `tcp/5432` en exposición, la dirección de origen en las reglas de autenticación, la ruta en las de integridad. Las reglas que solo pueden ocurrir una vez por activo (reloj, actualizaciones, concentración, salida) usan `key` vacío.

Consecuencia en la base de datos: `incidents.fingerprint` tiene restricción `UNIQUE`, así que la deduplicación entre ciclos es una propiedad del esquema, no solo del código.

`exposure::detect` — bucle sobre los sockets expuestos

Para cada socket que **no** es loopback:

1. Si el alcance es `Public` y la política lo permite para ese rol → `continue`.
2. Calcula clase y severidad. Si la severidad es `Info` → `continue`.
3. Elige la regla: `exposure.cleartext.protocol` si la clase es `Cleartext`, `exposure.service.public` en el resto.

4. Construye título, resumen, causa raíz y evidencia con textos distintos por regla. El resumen del protocolo en claro dice explícitamente que *quien observe la red obtiene las credenciales sin explotar ninguna vulnerabilidad*.
5. Adjunta el runbook de `runbook::restrict_listener` para la plataforma del activo, y confianza `0.99`.

Regla de negocio: la confianza es alta porque el hecho es directamente observable —el socket está o no está—. Compárese con `0.55` de la regla de salida anómala, donde la interpretación es incierta.

`intrusion::authentication` — de eventos a presión

Transformación inicial. Un bucle agrupa los `AuthEvent` por dirección de origen en un `BTreeMap<&str, SourcePressure>`, acumulando fallos, éxitos, nombres de usuario distintos (`BTreeSet`), servicios y el `last_seen` máximo. `saturating_add` protege de desbordamientos con contadores manipulados.

Después, por cada origen con al menos un fallo, se evalúan **tres reglas independientes** —pueden dispararse las tres a la vez—:

Condición	Regla	Severidad
`failures ≥ failures_per_source_high`	`intrusion.auth.bruteforce`	`High`, o `Critical` si supera `failures_per_source_critical`
`usernames.len() ≥ sprayed_usernames`	`intrusion.auth.spray`	`High`
`successes > 0` **y** `failures ≥ failures_per_source_high`	`intrusion.auth.success_after_burst`	`Critical` siempre

La tercera es el corazón del producto: **cruza dos hechos que ningún registro guarda juntos**. Su resumen instruye a tratar la sesión como comprometida hasta demostrar lo contrario, y sus acciones recomendadas incluyen buscar persistencia creada después del acceso.

Fuera del bucle, si el número de orígenes con fallos alcanza `distributed_sources`, se emite `intrusion.auth.distributed` con una observación operativa: *bloquear direcciones una por una no contendrá una campaña distribuida*.

`intrusion::egress` — línea base propia del host

Es la regla con más precondiciones, y todas son defensivas:

1. Necesita al menos **3 muestras** de historial; con menos, no hay línea base.
2. Calcula la tasa actual entre la última muestra histórica y la actual.
3. Si la tasa actual está por debajo de `egress_floor_bytes_per_second` (1 MB/s por omisión) → no evalúa. Sin suelo, un servidor ocioso dispararía la regla al mover unos pocos kilobytes.
4. Construye la línea base como media de las tasas entre pares consecutivos del historial.
5. Umbral = `max(baseline × egress_multiplier, suelo)`.
6. Severidad `High` si la tasa triplica el umbral; `Medium` si no.

En `egress_rate` hay un detalle importante:

```
let delta = current.network_tx_bytes.checked_sub(previous.network_tx_bytes)?;
```

Un contador que retrocede significa que la interfaz o el host se reiniciaron. `checked_sub` devuelve `None` y la regla se salta ese par en lugar de inventar una cifra enorme.

Confianza declarada: 0.55. El propio resumen dice que un respaldo, una réplica o una sincronización lo explican, y una exfiltración también.

`integrity::detect` — dos reglas por archivo

Para cada archivo vigilado:

- **Cambio de contenido.** Solo si existe una entrada previa en `file_baseline` y el digest difiere. La severidad sube a `High` cuando la ruta contiene uno de los 10 marcadores de `ACCESS_CRITICAL` (`sshd_config`, `sudoers`, `passwd`, `shadow`, `authorized_keys`, `pam.d`…); si no, `Medium`. La evidencia muestra los digests **acortados**, nunca el contenido.
- **Permisos debilitados.** `file.is_world_writable()` comprueba el bit `0o002`. Severidad `High`, confianza `0.99`.

Caso límite explícito: el primer ciclo de un archivo nunca genera hallazgo, porque crea la línea base. La prueba de integración `a_changed_watched_file_is_detected_only_after_a_baseline_exists` lo verifica.

`hygiene::detect` y `hygiene::clock`

- **Firewall.** Se dispara si `!enabled` **o** `!default_inbound_deny`. Severidad `High` si está desactivado, `Medium` si está activo pero acepta entrante por omisión. Entre las acciones recomendadas hay una que revela experiencia operativa: *asegura la regla de tu propio acceso remoto antes de activarlo, para no quedar fuera del servidor*.
- **Actualizaciones.** Solo si `pending_security_updates` es `Some(n)` con `n > 0`. Un `None` significa «no se pudo consultar» y **no genera nada**: ahí el agente ya emitió una brecha.
- **Reloj.** Corre **antes** de comprobar si hay superficie, porque solo necesita la diferencia entre `received_at` y `observed_at`. Si el valor absoluto supera `clock_skew_seconds`, emite el hallazgo indicando si el reloj va atrasado o adelantado.

`resource::cpu` — una racha, no un pico

```
let streak = sustained_samples(input.history, policy.resource.cpu_high) + 1;
if streak < required { return; }
```

`sustained_samples` cuenta, **desde el final del historial hacia atrás**, cuántas muestras consecutivas ya estaban por encima del umbral; el `+1` incluye la muestra actual. El comentario del código explica la razón: *un pico no es saturación, y reportarlo como tal es como una consola enseña a sus operadores a ignorarla*.

La confianza también depende de la racha: `0.92` si duplica lo requerido, `0.78` si apenas lo alcanza.

`resource::hours_to_full` y `runway`

```
Some(remaining / (growth / elapsed_hours))
```

Proyección lineal simple: crecimiento observado entre la primera muestra del historial y la actual, extrapolado al espacio restante. Devuelve `None` si el tiempo transcurrido no es positivo o si el disco no creció, y `Some(0.0)` si ya no queda margen.

`runway` añade dos guardas: exige 3 muestras y **se calla si la regla de capacidad ya está reportando** (`disk_percent ≥ disk_critical`), para no duplicar el mismo problema con dos hallazgos.

`availability::agent_silence` — la regla que corre en el servidor

Es la única que se dispara por **ausencia** de evidencia, y por eso no participa en `analyze`: un agente silenciado no puede reportar que fue silenciado.

```
let interval  = interval_seconds.max(5);
let tolerated = interval.saturating_mul(u64::from(policy.hygiene.silence_intervals));
let silence   = (now - last_seen).num_seconds();
if silence <= 0 || (silence as u64) < tolerated { return None; }
```

Severidad `High` si el silencio cuadruplica la ventana tolerada, `Medium` si no. El resumen es deliberadamente honesto: *un apagado planificado, una caída de red y un agente detenido a propósito se ven exactamente igual desde aquí.*

---

4 · `rootcause-core::posture` — un número con su letra pequeña

Entrada: los incidentes de un ámbito, más la última superficie observada (opcional). **Salida:** `PostureScore` con puntuación, nota, seis dimensiones y la lista de superficies no inspeccionadas.

Flujo:

1. Filtra los incidentes **no resueltos**.
2. Para cada una de las seis categorías: penalización = suma de pesos por severidad (45 crítico, 25 alto, 10 medio, 4 bajo, 0 informativo); puntuación de la dimensión = `100 − penalización`, con `saturating_sub` para no bajar de cero.
3. Media ponderada con pesos por categoría: intrusión y exposición 5, integridad y disponibilidad 3, higiene 2, recursos 1.
4. **Techo.** Aquí está la decisión de producto:

``rust let ceiling = open.iter().map(|i| i.severity).max().map_or(100, |worst| match worst { Severity::Critical => 55, Severity::High => 74, Severity::Medium => 89, _ => 100, }); let score = (u32::from(average) * ceiling / 100) as u8; ``

Sin el techo, una media ponderada diluiría una emergencia entre seis dimensiones y un servidor con una base de datos expuesta podría sacar «B» porque todo lo demás está en orden. Con el techo sigue puntuando peor cuantas más dimensiones estén afectadas, pero nunca puede parecer sano.

1. **Honestidad.** `uninspected_surfaces` se rellena con las brechas de recolección, formateadas como `superficie: razón`.

Notas: A ≥ 90, B ≥ 80, C ≥ 70, D ≥ 60, F por debajo.

---

5 · `rootcause-core::runbook` — escribir, nunca ejecutar

`RunbookStep` declara `kind` (inspeccionar / contener / corregir), plataforma, comando opcional, si necesita privilegios y si es reversible.

Los cuatro generadores siguen el mismo patrón:

1. **Primer paso siempre** `advice`, sin comando: *confirma que la dirección no es legítima antes de bloquearla.*

2. **Paso de inspección** con el comando concreto de la plataforma (`journalctl … --grep` , `Get-WinEvent -FilterHashtable`).
3. **Paso de contención** cuyo texto **incluye cómo revertirlo**: `ufw deny from … (revierte con: ufw status numbered && ufw delete <n>)` .
4. **Último paso** `advice` : registrar el bloqueo con fecha y responsable, porque *un bloqueo sin registro es indistinguible de un corte*.

La garantía la impone una prueba del propio módulo, `no_destructive_commands` : si alguien introduce un comando destructivo, **el build falla**. Es la traducción a código de la promesa «ninguna acción destructiva automática» de `AGENTS.md` .

---

6 · `rootcause-server::defense` — el perímetro propio

Por qué está en memoria

El comentario del módulo lo justifica: *el estado del perímetro debe sobrevivir a una ráfaga, no a un reinicio, y un reinicio es exactamente cuando un operador quiere empezar de cero*. Es una decisión, no una limitación.

`check` — cubeta de tokens, paso a paso

1. **Desalojo.** Si la tabla llegó a `MAX_TRACKED_CLIENTS` (20 000) y la dirección es nueva, se desaloja la entrada vista hace más tiempo. Sin esto, un atacante rotando direcciones agotaría la memoria del proceso.
2. **Bloqueo vigente.** Si `locked_until > now` → `LockedOut` . Si expiró, se **perdona el contador** de fallos y se sigue vigilando.
3. **Recarga.** Los tokens se rellenan según el tiempo transcurrido: `tokens + elapsed × (capacidad / 60)` , con tope en la capacidad.
4. **Consumo.** Si quedan menos de 1.0 tokens → `RateLimited` , con `retry_after_seconds` calculado a partir de lo que falta. Si no, resta uno y devuelve `Allow` .

`record_failure` incrementa el contador y activa el bloqueo al alcanzar el umbral, devolviendo `true` **solo la primera vez** —para que el registro se escriba una vez, no en cada intento posterior—. `record_success` limpia el contador.

---

7 · `rootcause-server::auth` — el orden es la característica

```
let decision = state.perimeter.check(client, now);
if !decision.is_allowed() { /* 429 */ }
let Some(expected) = state.api_token.as_deref() else { return Ok(next.run(request).await) };
let supplied = bearer_token(request.headers()).unwrap_or_default();
if !tokens_match(supplied, expected) { /* 401 + record_failure */ }
```

`client_address` **.** Solo lee `X-Forwarded-For` cuando el operador declaró que hay un proxy delante, y toma **la entrada más a la izquierda**, que es el cliente original. Si el valor es ilegible, vuelve a la dirección del par. Sin información de par, usa `0.0.0.0` como marcador estable —así una conexión sin `ConnectInfo` se maneja en lugar de rechazarse—.

`bearer_token`. Solo acepta el esquema `Bearer`; `Basic` devuelve `None`, y un `Bearer` seguido solo de espacios también.

`tokens_match`. Compara longitudes primero y después usa `subtle::ConstantTimeEq`. La comprobación de longitud filtra el tamaño del token, no su contenido.

Modo sin token. Cuando `api_token` es `None`, la petición pasa. Es seguro porque `ServeSettings::validate` garantiza que esa combinación solo existe escuchando en loopback.

---

8 · `rootcause-server::api::ingest_telemetry` — el ciclo completo

Es la función más larga del servidor y merece el recorrido íntegro.

Fase 1 — validación de entrada.

1. `protocol_is_supported(&envelope.protocol_version)` → si no, `400` diciendo qué versión habla este servidor.
2. `envelope.sample.validate()` → porcentajes finitos entre 0 y 100.
3. **Ventana temporal:** se rechaza una muestra con `observed_at` más de 5 minutos en el futuro o más de 24 horas en el pasado. Evita que un reloj roto o un reenvío masivo contamine el historial.
4. Si viene registro: `validate_asset` (nombre de equipo 1–255, versión y arquitectura no vacías y ≤ 50, ≤ 32 etiquetas con claves ≤ 64 y valores ≤ 256) y comprobación de que `asset.agent_id == sample.agent_id`.
5. `asset_registration(agent_id)` → si no existe, `404`: *registra el activo antes de enviar telemetría*.
6. Si viene superficie: `validate_security` acota tamaños (1 024 sockets, 4 096 pares, 1 024 eventos, 512 archivos, 64 brechas) y longitudes de campo. Sin esto, un agente comprometido podría inundar el plano de control.

Fase 2 — persistencia de la evidencia.

`store_sample`, y si hay superficie también `store_security` y `record_auth_pressure`. Cada brecha de recolección se convierte en una advertencia de la respuesta. Si **no** hay superficie, se añade la advertencia *«solo se evaluaron los recursos»*.

Fase 3 — contexto para las reglas.

`recent_samples(agent_id, observed_at, 12)` y `file_baseline(agent_id)`. Doce muestras son el `HISTORY_WINDOW`: suficiente para una racha de CPU y para una línea base de salida, poco para que la consulta pese.

Fase 4 — detección y escritura.

```
let input = DetectionInput::new(&registration, &envelope.sample, received_at, &file_baseline)
    .with_security(envelope.security.as_ref())
    .with_history(&history);
let candidates = state.engine.analyze(&input);
```

Cada candidato pasa por `upsert_incident`, y su huella se guarda en un `BTreeSet` que servirá para el paso siguiente.

Fase 5 — auto-resolución honesta.

```
fn self_healing_categories(security: &SecuritySignals) -> Vec<Category> {
    let mut categories = Vec::new();
    if !security.has_gap("listeners") { categories.push(Category::Exposure); }
    if !security.has_gap("firewall") && !security.has_gap("security-updates") {
        categories.push(Category::Hygiene);
    }
    categories
}
```

Esta función es REQ-SEC-003 hecha código. Solo puede cerrarse sola una categoría cuya superficie se inspeccionó de verdad. Cerrar un hallazgo de firewall porque el agente no pudo leer el firewall sería reportar un cero donde nadie miró.

Nótese que las categorías `Intrusion`, `Integrity`, `Availability` y `Resource` **nunca** se auto-resuelven aquí: describen algo que *ocurrió*, y un hecho pasado no deja de haber ocurrido porque el ciclo siguiente esté tranquilo.

Fase 6 — cierre.

`update_file_baseline` (después de detectar, nunca antes, o el cambio se perdería), recálculo de postura del activo, `store_posture`, y respuesta con `accepted`, `incidents_touched` y las advertencias.

---

9 · `rootcause-server::storage` — SQLite con la evidencia intacta

`connect`

Detecta si la URL es en memoria; activa `create_if_missing`, claves foráneas, espera de 10 s y `synchronous = NORMAL`; usa WAL salvo en memoria; abre hasta 8 conexiones (1 en memoria, porque una base en memoria por conexión sería una base distinta cada vez); ejecuta `sqlx::migrate!("./migrations")`.

`upsert_incident` — las reglas de negocio del ciclo de vida

Consulta por huella. Si existe:

```
incident.occurrences = incident.occurrences.saturating_add(1);
incident.status   = IncidentStatus::Open;   // reconocer no es arreglar
incident.severity = incident.severity.max(candidate.severity);
```

Tres reglas de negocio en tres líneas:

1. **Un hallazgo que se repite es el mismo hallazgo**, con más ocurrencias.
2. **Reabre aunque estuviera reconocido.** Reconocer no es arreglar: si la condición sigue ocurriendo, vuelve a la bandeja.
3. **Conserva la severidad máxima vista.** Un incidente que llegó a crítico no se degrada porque un ciclo posterior lo viera menos grave.

Los demás campos (título, resumen, causa, evidencia, runbook) **sí** se refrescan con lo último observado.

`auto_resolve`

Recorre los incidentes no resueltos del activo; salta los que siguen presentes en el conjunto de huellas de este ciclo; salta los de categorías que no se pasaron; marca el resto como `resolved` y **escribe una entrada de auditoría** con el motivo. El cierre queda registrado, no ocurre en silencio.

`list_incidents`

Construye el SQL de forma incremental (`WHERE 1 = 1` y `AND` según los filtros presentes) y **enlaza los valores como parámetros**, nunca por interpolación. No hay concatenación de datos de usuario en la consulta.

`purge`

Borra de `telemetry` , `auth_pressure` y `defense_events` lo anterior al corte. **No borra incidentes ni auditoría**: esos son la evidencia, y la evidencia no caduca por omisión.

`threat_severity` — la escala de la vista de amenazas

```
if successes > 0 && failures >= 20 { return Severity::Critical; }
match failures { 0..=4 => Info, 5..=19 => Low, 20..=99 => High, _ => Critical }
```

La primera línea repite en el informe de flota la misma correlación que la regla `intrusion.auth.success_after_burst` aplica por activo.

---

10 · `rootcause-server::watchdog` — lo que pasa cuando nadie habla

Bucle infinito con `sleep(TICK)` de 30 s. En cada vuelta:

1. `sweep_silence` — pide los latidos, evalúa `agent_silence` por activo y, o bien crea el incidente, o bien **auto-resuelve la categoría** `Availability` para el activo que volvió a reportar.
2. `perimeter.prune(now, PERIMETER_IDLE)` — olvida direcciones inactivas 900 s.
3. Cada 120 ticks (≈ 1 hora), `purge(retention_days)` .

En `sweep_silence` hay un `debug_assert_eq!` que comprueba que la huella del silencio sigue siendo `"{agent_id}:availability.agent.silence"` . Es una prueba de estabilidad de formato: si alguien cambia el formato de la huella, los incidentes viejos dejarían de coincidir con los nuevos y no se cerrarían nunca.

Tratamiento de errores: cada paso captura su fallo, lo registra con `warn!` y continúa. Una lectura fallida de latidos no debe impedir la retención.

---

11 · `rootcause-server::headers` y `ui` — la consola no se puede armar

`headers::apply` inserta ocho cabeceras en toda respuesta, añade HSTS **solo** si `X-Forwarded-Proto: https` demuestra que la petición llegó por TLS, y pone `Cache-Control: no-store` en las rutas bajo `/api/` porque la evidencia no debe servirse desde una caché compartida.

La CSP es `script-src 'self'` sin `unsafe-inline` . Esa única decisión es la razón por la que `console/app.js` construye cada nodo con la API del DOM en lugar de con plantillas: un nombre de equipo o de proceso reportado por un agente **no puede** interpretarse como marcado.

`ui::static_asset` protege las rutas reservadas (`api/`, `healthz`, `readyz`, `metrics`) devolviendo `404` antes de tocar los activos, y sirve `index.html` para cualquier ruta desconocida, de modo que la navegación del lado del cliente funcione. Como los activos van embebidos con `rust-embed`, no hay travesía de rutas posible: lo que se sirve es exactamente lo que se compiló.

---

12 · `rootcause-agent::probe` — la promesa auditable

El módulo entero existe para que la afirmación *«el agente no modifica el host»* se pueda comprobar **leyendo un solo archivo**.

```
pub fn is_allowed(program: &str) -> bool {
    let name = program.rsplit(['/', '\\']).next().unwrap_or(program);
    let name = name.strip_suffix(".exe").unwrap_or(name);
    ALLOWED_PROGRAMS.iter().any(|(allowed, _)| *allowed == name)
}
```

Normaliza rutas absolutas y el sufijo `.exe` antes de comparar, de modo que `/usr/bin/ss` y `C:\Windows\System32\netstat.exe` se resuelven contra la misma lista de 11 programas.

`run` comprueba la lista **antes** de tocar el sistema operativo, anula `stdin`, descarta `stderr`, activa `kill_on_drop` y envuelve la ejecución en `tokio::time::timeout`. Cada modo de fallo tiene su variante de `ProbeError` y su explicación textual para el operador, que acaba tal cual en una `CollectionGap`.

---

13 · `rootcause-agent::net` — tres plataformas, un contrato

`collect()` elige el recolector según el sistema operativo y después `truncate()` recorta a 256 sockets y 512 pares, **añadiendo una brecha que dice que se recortó**. Un host bajo escaneo puede tener decenas de miles de conexiones; enviarlas todas convertiría al sensor en el incidente.

En Linux hay una cadena de respaldo: si `ss` no está disponible, lee `/proc/net/tcp`, `tcp6`, `udp`, `udp6` directamente; y solo si tampoco eso produce nada, registra la brecha.

Los cuatro parsers (`parse_ss`, `parse_netstat_windows`, `parse_netstat_bsd`, `parse_proc_net`) son **funciones puras** sobre texto. Esa pureza es lo que permite ejecutar en CI, sobre cualquier sistema operativo, el código que interpreta la salida de herramientas de otros tres.

Detalles que valen la pena:

- `normalise_wildcard` traduce las formas de «todas las interfaces» de cada herramienta a una representación común.
- `parse_proc_address` decodifica las direcciones hexadecimales de `/proc`, convirtiendo el orden de bytes palabra a palabra de 32 bits (`to_le_bytes`).
- `aggregate_peers` colapsa las conexiones repetidas de un mismo origen hacia un mismo puerto local incrementando `connections`, en vez de enviar una fila por conexión.

---

14 · `rootcause-agent::authlog` — contar sin copiar

Objetivo declarado: lo que viaja al servidor es un agregado (`service` , `source` , `user` , `outcome` , `count`), **nunca una línea de registro**. Las cuentas son lo que la detección necesita; las líneas crudas son lo que se filtraría.

En Linux usa `journalctl` con `--since -Nmin` acotado por `--auth-window-minutes` , y si falla lee `/var/log/auth.log` o `/var/log/secure` quedándose **solo con las últimas 5 000 líneas**: la ventana se mide en minutos, no en meses.

En Windows consulta `wevtutil` para los eventos 4625 (fallo) y 4624 (éxito).

`aggregate` colapsa las filas por la tupla (servicio, origen, usuario, resultado) y recorta a `MAX_EVENTS` (200) para dejar la lista de los mayores ofensores sin inflar el sobre.

---

15 · `rootcause-agent::integrity` — huella sí, contenido no

`collect` recorre la lista de vigilancia y distingue tres desenlaces, que es justo la distinción que importa:

Desenlace	Significado	Qué produce
`Ok(Some(file))`	Se leyó y se calculó la huella	Un `WatchedFile`
`Ok(None)`	El archivo **no existe**	Nada. La mayoría de servidores no ejecutan nginx
`Err(reason)`	Existe pero no se pudo leer	Una `CollectionGap`

El tercer caso es el que evita que un operador asuma una cobertura que no tiene. Un `/etc/shadow` sin permiso de lectura aparece como brecha, no como «sin cambios».

Además hay un tope de 8 MiB: un archivo de configuración nunca es tan grande, y calcular la huella de un registro de varios gigabytes en cada ciclo *sería* el incidente.

---

16 · `rootcause-agent::main` — transporte y reintento

Antes de nada: `validate_transport` y una comprobación de longitud mínima del token. El agente prefiere no arrancar a poner una credencial en un canal inseguro.

`--dry-run` construye el sobre completo y lo imprime formateado, sin enviarlo. Es la forma de auditar exactamente qué saldría del equipo **antes** de enrolarlo.

`send_once` registra el activo y después envía la telemetría; ambas llamadas usan `error_for_status()` , de modo que un `4xx` o `5xx` se convierte en error de Rust con contexto legible. Las advertencias que devuelve el servidor se registran una por una con `warn!` .

Bucle principal. Reintento exponencial: el intervalo se duplica hasta `MAX_BACKOFF` (300 s) tras cada fallo y vuelve al normal en cuanto un envío tiene éxito. La espera y `ctrl_c` compiten en un `tokio::select!` , así que el agente responde a la señal de inmediato en lugar de esperar al final del intervalo.

---

17 · `console/app.js` — construir el DOM a mano

Restricción de partida: la CSP prohíbe script en línea. La consecuencia está escrita en la cabecera del archivo: cada nodo se construye con la API del DOM porque *un nombre de equipo o de proceso reportado por un agente nunca puede interpretarse como marcado*.

Estructura:

- `state` — objeto único con token, respuestas de cada endpoint, filtros y el temporizador.
- `el()` y `svg()` — pequeños ayudantes de creación de nodos. Todo el renderizado pasa por ellos.
- `api()` y `post()` — envoltorios de `fetch` con `cache: "no-store"`. Un `401` abre el diálogo del token; un `429` lee `Retry-After` y lo muestra en el mensaje de error.
- `refresh()` — pide todos los endpoints y vuelve a renderizar; se repite cada 10 s.
- Un `render*()` por vista: postura, métricas, endurecimiento, incidentes, superficie, amenazas, activos, topología, reglas, sistema y auditoría.
- `layoutTopology()` — distribuye los nodos y dibuja el SVG sin biblioteca externa.

El token se guarda en `sessionStorage`, no en `localStorage`: se pierde al cerrar la pestaña. El botón «modo local» lo borra para trabajar contra una instancia sin autenticación.

---

18 · Los guardianes de `scripts/`

Script	Lo que impide	Cómo
`guard_claims.py`	Que la documentación afirme un número que el código ya no cumple, y que una acción de GitHub quede en una etiqueta móvil	Cuenta las entradas de `RULES` y compara con los documentos; exige SHA de 40 caracteres en cada `uses:`
`guard_console.py`	Que alguien añada «solo por esta vez» un manejador en línea, un enlace a CDN o un `innerHTML`	Expresiones regulares sobre `console/`, más comprobación de la CSP en `headers.rs`
`guard_encoding.py`	Que un archivo escrito una vez en Latin-1, con BOM o con CRLF acabe mostrando una palabra rota a la persona a la que intenta advertir	Recorre 109 archivos de texto y valida codificación, BOM, saltos de línea y patrones de mojibake

El razonamiento común está escrito en `guard_claims.py`: los números de un README se desincronizan en cuanto alguien añade una regla y olvida el documento, así que **la cifra se deriva de la fuente de verdad y el build falla cuando las dos no coinciden**. «18 reglas» es un hecho comprobado, no un recuerdo.

Anterior: 05 · Referencia técnica · **Siguiente:** 07 · Base de datos · **Índice:** README
